

PROJECT REPORT ON
“HEALTHCARE INSURANCE
MANAGEMENT SYSTEM (HIMS)”

A report submitted in partial fulfillment of the requirements for

INTEGRATED PROFESSIONAL CORE COURSE-
DATABASE MANAGEMENT SYSTEMS (AD2102-1)

Submitted by

Abhijna N	NNM22AD002
Ayush Chaudhary	NNM22AD015
Isha Uday Naik	NNM22AD022
Krishna N Acharya	NNM22AD026

Under the Guidance of

Mr. Prajwal Hegde N

Assistant Professor Gd-II

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that the Integrated Professional Core Course project entitled “HEALTHCARE INSURANCE MANAGEMENT SYSTEM (HIMS)” report has been submitted by Abhijna N(NNM22AD002), Ayush Chaudhary(NNM22AD015), Isha U Naik(NNM22AD022) and Krishna N Acharya(NNM22AD026) of II year B. Tech, bonafide|students of NMAM Institute of Technology, Nitte, has completed the project work during the academic year 2023-2024 fulfilling the partial requirements of Integrated Professional Core Course lab in Artificial Intelligence & Data Science at NMAM Institute of Technology, Nitte.

Project Guide

Abstract

This project details the design and documentation of a Healthcare Insurance Management System (HIMS) using a Database Management System (DBMS). The HIMS aims to automate and streamline various healthcare insurance operations, improving efficiency and accuracy in managing policyholders, claims, medical records, and billing processes.

The report outlines the system's functionalities, including managing person and policy details, processing claims, conducting medical tests, generating bills, and maintaining hospital information. It further delves into the database design, explaining the structure of tables, data types, and relationships between them to support the functionalities.

Key components of the system include the Person table for managing personal information, the Policy table for policy details, the Disease table for recording medical conditions, the Claim table for processing claims, the Test table for medical tests, the Billing table for managing billing information, the Address table for addresses, and the Hospital table for hospital details. Implementation details cover the process of creating the database schema and populating it with sample data.

The report concludes by evaluating the effectiveness of the developed HIMS, discussing achieved functionalities, potential limitations, and opportunities for future enhancements. Overall, the project showcases the practical application of DBMS concepts in a real-world healthcare insurance scenario, emphasizing automation, data integrity, and efficient management of healthcare-related information.

Acknowledgement

Our Healthcare Insurance Management System (HIMS) project represents a collaborative effort involving dedicated team members and invaluable support from mentors and resources. We express our gratitude to Mr. Prajwal Hegde N, Assistant Professor Gd-II, Department of Artificial Intelligence and Data Science, for their guidance and expertise throughout the development process.

The HIMS project provided us with hands-on experience in designing and implementing solutions tailored to the healthcare and insurance industries' unique needs. Focused on optimizing data management, enhancing security protocols, and improving user experience, the system aims to revolutionize healthcare insurance management.

With features such as policy management, claims processing, medical records handling, and billing management, the HIMS project promises to streamline insurance operations and improve healthcare delivery for policyholders. We eagerly anticipate the implementation of the system and its positive impact on healthcare efficiency and customer satisfaction.

Contents

Certificate	ii
Abstract.....	iii
Acknowledgement.....	iv
Chapter 1- Introduction	1
1.1 Overview	1
1.2 Objectives.....	1
1.3 Importance.....	1
Chapter 2- Problem Statement.....	2
Chapter 3-Glossary.....	3
Chapter 4- User Requirements	4
4.1 Functional requirements.....	4
4.2 Non-Functional requiremnts	5
Chapter 5- Methodology	6
5.1 Model Used	9
5.2 Relationship Cardinality.....	10
Chapter 6- System Model.....	13
Chapter 7- System Architechture	14
Chapter 8- Data Overflow Overview.....	15
Chapter 9- Use Case Overview	16
Chapter 10- System Evolution.....	30
Chapter 11- Testing.....	37

Chapte 12- Results	44
Chapter 13- Conclusion	49

List of tables

Table 5.1 Relationship Cordinality	12
Table 9.1 Enroll in policy.....	18
Table 9.2 Manage Policy.....	19
Table 9.3 Submit Claim.....	20
Table 9.4 Admit Patient.....	21
Table 9.6 Generate Bills.....	22
Table 9.7 Verification Insurance Coverage	22
Table 9.8 Update Patient Medical Records.....	23
Table 9.9 Process Enrolment	24
Table 9.10 Manage Policies.....	25
Table 9.11 Process Claims	26
Table 11.1 Test Cases	37
Table 12.1 Output 1	44
Table 12.2 Output 2	45
Table 12.3 Output 3	45
Table 12.4 Output 4	46
Table 12.5 Output 5	47

Table 12.6 Output 6	48
---------------------------	----

List of Diagrams

Figure 6.1 ER Diagram.....	13
Figure 7.1 Schema Diagram	14
Figure 7.1 Data Overflow Diagram	15
Figure 9.1 Use Case Diagram.....	16

Chapter 1

Introduction

1.1 Overview

The Healthcare Insurance Management System (HIMS) project endeavors to modernize and optimize healthcare insurance operations significantly. Through the implementation of a robust DBMS-based solution, HIMS aims to revolutionize operations, ensuring enhanced efficiency and transparency by addressing challenges such as complex policy management and inefficient claims processing. By leveraging advanced technology and data management techniques, HIMS seeks to establish a seamless and streamlined healthcare insurance ecosystem.

1.2 Objectives

HIMS is strategically designed to automate policy management tasks, optimize claims workflows, and enhance data accessibility across the system. Real-time updates and seamless communication integration are fundamental aspects of HIMS, ensuring timely decision-making and improved collaboration among stakeholders. Furthermore, the implementation of robust security measures is crucial to safeguard sensitive data and ensure compliance with regulatory standards.

1.3 Importance HIMS holds immense importance in streamlining processes, improving data accuracy, and fostering better communication among stakeholders. The anticipated benefits of HIMS encompass faster policy issuance, efficient claims processing, and elevated customer service standards, ultimately creating a highly responsive and secure healthcare insurance environment.

Chapter 2

Problem Statement

In the healthcare industry, there is a need for a centralized and efficient database management system to handle the complex data requirements involving patient information, insurance policies, medical records, claims processing, billing, and hospital data. Current systems often lack scalability, performance, data integration capabilities, and fail to meet regulatory compliance standards. The lack of a robust DBMS leads to operational inefficiencies, increased costs, and potential risks to patient safety and data privacy. Our project aims to develop a comprehensive DBMS that can seamlessly manage and integrate these diverse data entities, ensuring data integrity, security, and compliance while enabling real-time access to critical information for improved healthcare delivery.

Chapter 3

Glossary

SQL: SQL is a standard language for storing, manipulating and retrieving data in databases.

Database: A database is a collection of data that is stored electronically in a computer system, and is usually controlled by a database management system (DBMS).

DBMS: A Database Management System (DBMS) is a software system that stores, retrieves, and runs queries on data.

Software Design: Software design provides a design plan that describes the elements of a system, how they fit, and work together to fulfil the requirement of the system.

Incremental Agile Model: An iterative software development approach that emphasizes incremental delivery of working software, frequent collaboration with stakeholders, adaptability to changing requirements, and continuous improvement.

Incremental Development: The process of building a system in small, incremental steps, with each step delivering a functional subset of the overall system.

System model: In the context of software engineering, a system model refers to a representation or abstraction of a software system that helps stakeholders understand its structure, behavior, and interactions.

Agile manifesto: A statement on the values that underpin agile software development. The values are:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- responding to change over following a plan.

Architectural Design: Architectural design is the specification of the major components of a system, their responsibilities, properties, interfaces, and the relationships and interactions between them.

Functional Requirements: These are the requirements that the end user specifically demands as basic facilities that the system should offer. All these functionalities need to be necessarily incorporated into the system as a part of the contract.

Non-Functional Requirements: These are the quality constraints that the system must satisfy according to the project contract. The priority or extent to which these factors are implemented varies from one project to another. They are also called non-behavioral requirements.

Test-Driven Development (TDD): A development approach where automated tests are written before the corresponding code, guiding the design and ensuring that the code meets the specified requirements.

Sprint: A time-boxed iteration in Agile development, typically lasting 1-4 weeks, during which a set of user stories or tasks are completed and potentially shippable software increments are produced.

Sprint Backlog: A subset of items from the product backlog selected for implementation during a specific sprint, along with associated tasks and acceptance criteria.

Scrum Master: A facilitator and servant-leader for an Agile development team, responsible for ensuring adherence to Agile practices, removing impediments, and fostering a productive and collaborative environment.

Chapter 4

User Requirements

4.1 Functional requirements

Person Management:

- Add/Edit/Delete Person: Allow adding, editing, and deleting person records.
- Search/Filter Persons: Provide a search and filter functionality for easy retrieval of person records.
- View Person Details: Allow viewing detailed information about each person.

Policy Management:

- Create/Update Policy: Enable the creation and updating of insurance policies.
- View Policy Details: Allow users to view details of each policy, including sum insured, premium, benefits, etc.
- Terminate Policy: Provide functionality to terminate policies when needed.

Disease Management:

- Add/Edit/Delete Disease: Allow adding, editing, and deleting disease records.
- View Disease Details: Provide information about each disease, including code and name.

Claim Management:

- File Claim: Allow persons to file insurance claims.
- Process Claim: Enable processing of claims, including verifying details, calculating claimed and incurred amounts, and updating claim status.

Test Management:

- Add/Edit/Delete Test: Allow managing medical tests, including adding, editing, and deleting test records.
- View Test Details: Provide information about each medical test, including code, disease association, name, and price.

Billing Management:

- Generate Bill: Automatically generate bills based on medical services provided.
- View Billing Details: Allow viewing billing information, including admission and discharge dates, bill amount, disease code, etc.

Address Management:

- Add/Edit/Delete Address: Enable managing addresses for persons, hospitals, etc.
- View Address Details: Provide details such as state, city, and pin code for each address.

Hospital Management:

- Add/Edit/Delete Hospital: Allow managing hospital records.
- View Hospital Details: Provide information about each hospital, including name and address.

4.2 Non Functional Requirements:**Performance:**

- The DBMS should be able to handle a large volume of transactions and data processing without significant performance degradation.
- Query response times should be within acceptable limits, even during peak load periods.
- The system should be optimized for efficient data retrieval and storage operations.

Scalability:

- The DBMS should be capable of scaling horizontally (adding more servers) and vertically (upgrading hardware resources) to accommodate increasing data and user loads.
- It should support distributed architectures and data partitioning techniques to ensure scalability.

Availability and Reliability:

- The DBMS should have high availability, with minimal downtime and data loss.
- It should support failover mechanisms, redundancy, and backup strategies to ensure data integrity and business continuity.
- The system should be fault-tolerant and able to recover from hardware or software failures.

Security:

- The DBMS should implement robust security measures, such as authentication, authorization, and encryption, to protect sensitive data (e.g., medical records, personal information).
- It should comply with relevant data privacy regulations and standards (e.g., HIPAA, GDPR).
- Access controls and auditing mechanisms should be in place to track and monitor user activities.

Concurrency Control:

- The DBMS should support concurrent access to data by multiple users and transactions while maintaining data integrity and consistency.
- It should implement appropriate locking mechanisms or other concurrency control techniques to prevent conflicts and ensure data integrity.

Data Integrity:

- The DBMS should enforce data integrity constraints, such as referential integrity, entity integrity, and domain integrity, to ensure data accuracy and consistency.
- It should provide mechanisms for data validation, error handling, and rollback capabilities.

Backup and Recovery:

- The DBMS should support regular data backups and provide efficient recovery mechanisms in case of system failures or data loss.
- It should allow for point-in-time recovery and transaction log management for rollback and recovery purposes.

Interoperability:

- The DBMS should support integration with other systems and applications through standard interfaces and protocols (e.g., JDBC, ODBC).
- It should provide mechanisms for data exchange and synchronization with external systems or data sources.

Chapter 5

Methodology

5.1 Model used

Our approach to developing the insurance and healthcare management system embraces an **Agile Incremental methodology**, ensuring a dynamic and iterative process. We initiate the project by breaking down the functionalities into smaller, manageable features, such as person management, policy handling, claim processing, and billing management. Each feature undergoes continuous refinement and prioritization, reflecting evolving business needs and user feedback. Our development cycles, typically lasting 1-4 weeks, focus on delivering specific sets of features, which are thoroughly tested through automated testing protocols integrated into our continuous integration framework. Daily standup meetings foster collaboration and alignment within our cross-functional teams, comprising developers, testers, designers, and domain experts.

User involvement is paramount throughout the development lifecycle, with stakeholders regularly providing feedback during sprint reviews and user acceptance testing. This feedback loop ensures that the system meets user expectations and delivers tangible value with each incremental release. Our adaptive planning approach allows us to embrace changes in requirements and adjust sprint goals and backlog priorities accordingly, ensuring flexibility and responsiveness to evolving needs.

By combining Agile principles with incremental delivery, collaborative teamwork, user-centric design, and rigorous quality assurance, we ensure the development of a robust, efficient, and user-friendly insurance and healthcare management system that meets the diverse needs of our stakeholders.

The Iterative steps are:

1. Getting the Data Required.
2. Identifying the Entities, Attributes and the Relationship between each entity. Table 4.1 shows the entity with their respective attribute values.

3. Designing a Schema Diagram: Figure 1 in page no.7 shows the pictorial representation of the Schema Diagram.
4. Building the ER-Model: Figure 2 in page no.8 shows the pictorial representation of ER-Model.
5. Creating the Tables and Inserting Values: page no.9 shows the creation of the tables and inserting values.
6. Generating Relational Algebra Queries and SQL Queries: page no.14 contains the queries and the solution.

Person

- PersonID (INT - Primary Key)
- Name (VARCHAR(255))
- DOB (DATE)
- Status (VARCHAR(255))
- PhoneNumber (VARCHAR(20))
- Password (VARCHAR(255))
- Email (VARCHAR(255))
- Age (INT)

Address

- PersonID (INT - Foreign Key referencing Person)
- State (VARCHAR(255))
- City (VARCHAR(255))
- PinCode (VARCHAR(10))

Disease

- DiseaseCode (INT - Primary Key)
- DiseaseName (VARCHAR(255))

Hospital

- HospitalID (INT - Primary Key)
- HospitalName (VARCHAR(255))
 - HospitalAddress (VARCHAR(255))
 - State (VARCHAR(255))
 - City (VARCHAR(255))
- PinCode (VARCHAR(10))
- PinCode (VARCHAR(10))

Billing

- BillID (INT - Primary Key)
- DateOfDischarge (DATE)
- DateOfAdmission (DATE)
- BillAmount (DECIMAL(10,2))
- DiseaseCode (INT - Foreign Key referencing Disease)

Test

- TestCode (INT - Primary Key)
- TestName (VARCHAR(255))
- Price (DECIMAL(10,2))
- DiseaseCode (INT - Foreign Key referencing Disease)

Claim

- ClaimID (INT - Primary Key)
- PolicyNumber (INT - Foreign Key referencing Policy)
- PersonID (INT - Foreign Key referencing Person)
- ClaimedAmount (DECIMAL(10,2))
- IssuedAmount (DECIMAL(10,2))
- IssuedDate (DATE)
- Status (VARCHAR(255))

Policy

- PolicyNumber (INT - Primary Key)
- SumInsured (DECIMAL(10,2))
- Premium (DECIMAL(10,2))
- MedicalHistory (TEXT)
- PolicyBenefits (TEXT)
- Tenure (INT)

Table 5.1 Relationship Cordinity

Entity	Relationship	Related Entity	Cardinality
PERSON	stays at	ADDRESS	1-1
PERSON	buys	POLICY	1-N
PERSON	infected from	DISEASE	1-N
DISEASE	cured at	HOSPITAL	N-1
DISEASES	diagnosed by	TESTS	N-N
HOSPITAL	generates	BILLINGS	1-N
POLICY	provides	CLAIMS	1-N

Chapter 6

System Model

Figure 6.1 shows the design of the ER-Model for the system. There are 10 entities which are represented by the rectangle shape and their attributes represented in ellipse shape. Each entity is related to one or more entities which is represented by the diamond shape, the relations are as follows:

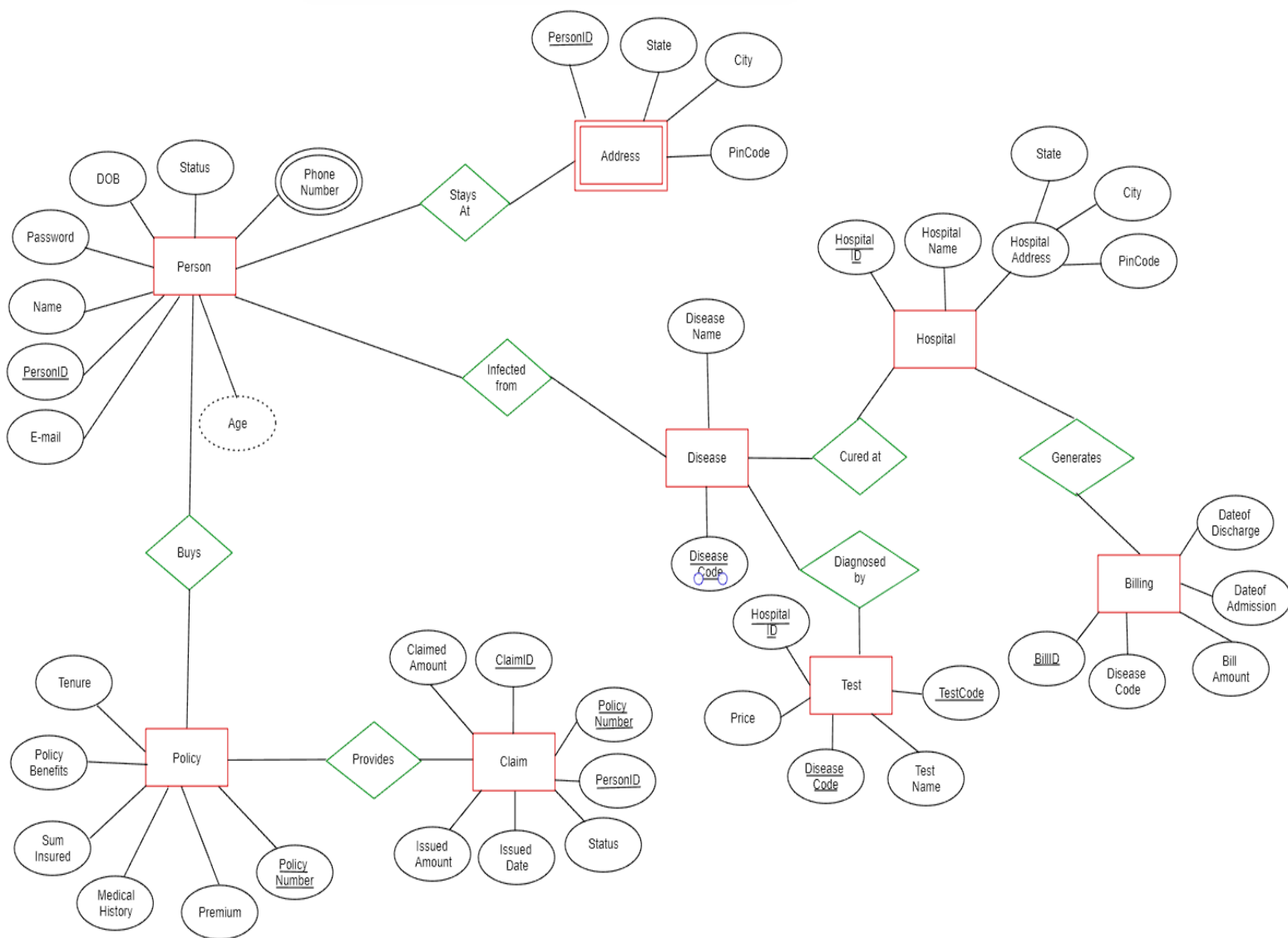


Figure 6.1 ER diagram

Chapter 7

System Architecture

A schema is the blueprint of a database, depicting the relationships between tables and supporting the database's mission and rules. It outlines entities and their connections, aiding in database comprehension and usage by programmers. Schema diagrams visually represent this structure, helping database administrators grasp the database's entirety and facilitating database development and maintenance. These diagrams use arrows to show connections between foreign keys and primary keys, simplifying data management and system understanding.

Figure 7.1 shows the pictorial representation of the schema of the Health Insurance Management System. The arrows indicates the connection between the foreign key and the primary key.

Person:

<u>Person ID</u>	Name	DOB	Status	PhoneNumber	Age	Password	Email
------------------	------	-----	--------	-------------	-----	----------	-------

Address:

State	City	PinCode	<u>Person ID</u>
-------	------	---------	------------------

Claim:

<u>Claim ID</u>	<u>PolicyNumber</u>	IssuedDate	ClaimedAmount	IncurredAmount	Status	<u>Person ID</u>
-----------------	---------------------	------------	---------------	----------------	--------	------------------

Policy:

<u>PolicyNumber</u>	SumInsured	Premium	MedicalHistory	PolicyBenefits	Tenure
---------------------	------------	---------	----------------	----------------	--------

Hospital:

<u>Hospital ID</u>	Name	Hospital Address
--------------------	------	------------------

Test:

<u>Test Code</u>	Disease Name	Test Name	Price	<u>Hospital ID</u>
------------------	--------------	-----------	-------	--------------------

Disease:

<u>Disease Code</u>	Disease Name
---------------------	--------------

Billing:

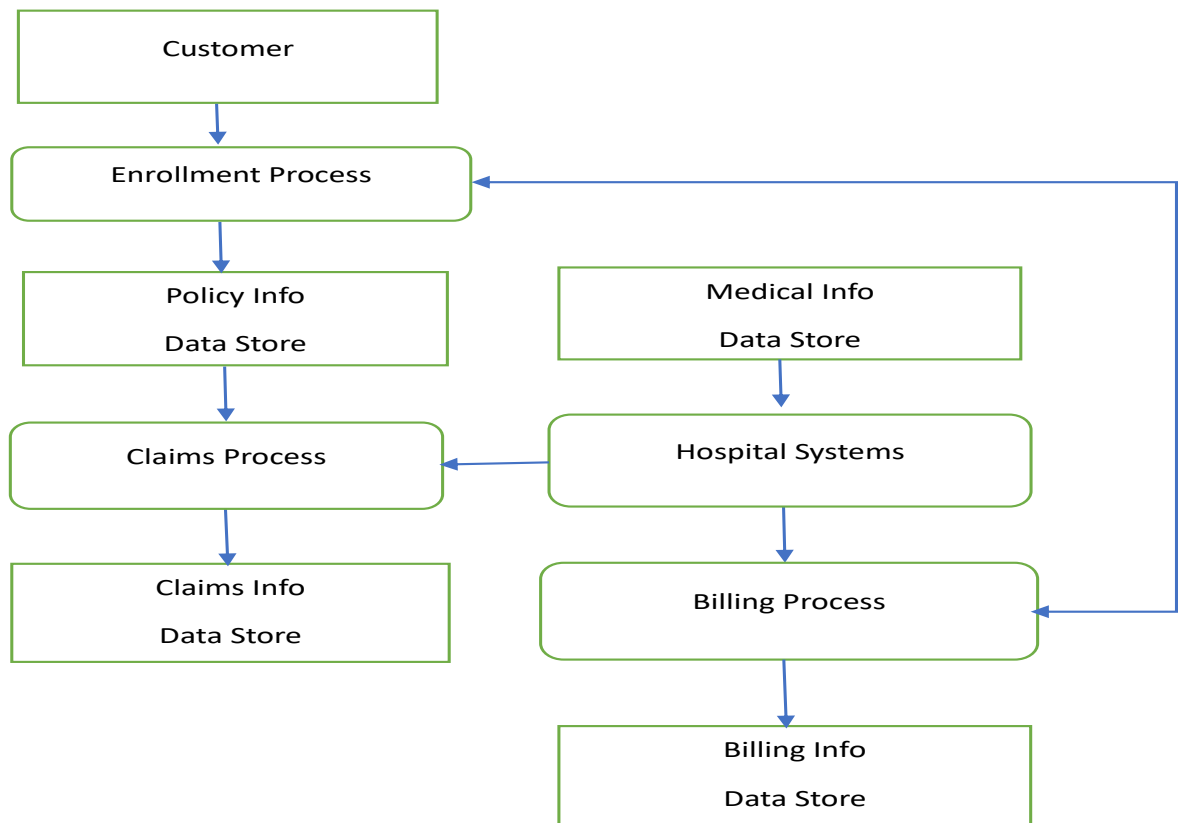
<u>Bill ID</u>	Date of Discharge	Date of Admission	Bill Amount	<u>Disease Code</u>
----------------	-------------------	-------------------	-------------	---------------------

Figure 7.1 Schema Diagram

Chapter 8

Data Flow Overview

Figure 8.1 Data Overflow Diagram



The data flow diagram (Figure 8.1) outlines key functions of a healthcare management system. It begins with patient registration, capturing demographics, medical history, and insurance details. Healthcare providers input clinical data like diagnoses, medications, and test results. This data then feeds into billing, calculating charges for services provided. Generated bills are sent to patients or insurance providers for claim processing, and payments update patient accounts. The system also generates reports for billing accuracy, monitoring hospital performance, and analyzing healthcare trends. This analysis offers insights into how the system supports patient care, billing, and reporting, enhancing hospital efficiency.

Chapter 9

Use Case Overview

9.1 Use Case Diagram:

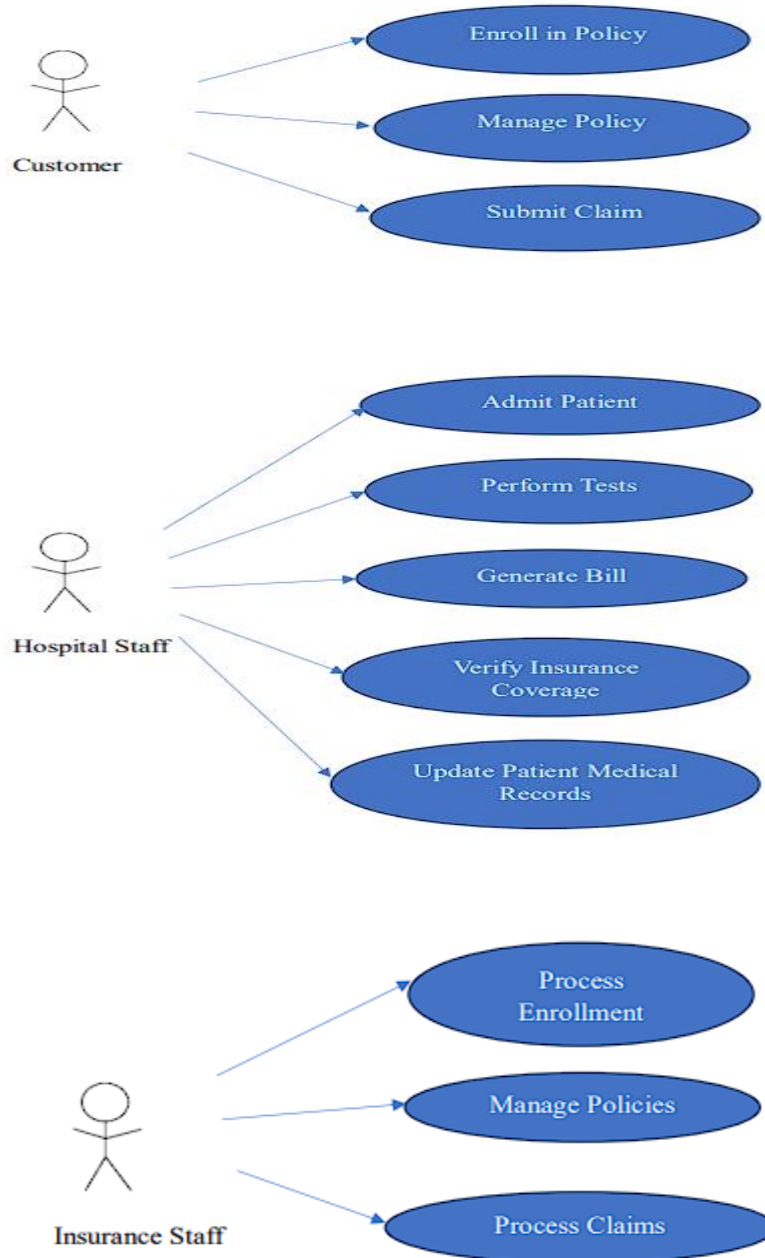


Figure 9.1 Use Case Diagram

In Figure 9.1, each interaction (use case) is represented by a line connecting the actor to the system, labeled with the action they perform.

Customer:

Enroll in Policy: This is where a customer signs up for a new insurance plan with the insurance company.

Manage Policy: This could involve tasks like updating personal information, making premium payments, or viewing policy details.

Submit Claim: This is what a customer does when they need to use their insurance, typically after receiving medical care.

Hospital Staff:

Admit Patient: This is the process of registering a patient for treatment at the hospital.

Perform Tests: This covers any medical examinations or procedures conducted on the patient.

Generate Bill: Once treatment is complete, the hospital creates a bill detailing the costs incurred.

Verify Insurance Coverage: The hospital checks with the insurance company to confirm the patient's coverage and benefits.

Update Patient Medical Records: This involves documenting the treatment provided to the patient.

Insurance Staff:

Process Enrollment: This refers to the insurance company's actions after a customer signs up for a policy, such as issuing policy documents and setting up accounts.

Manage Policies: This includes tasks like updating policy details, handling customer inquiries, and managing renewals.

Process Claims: This involves reviewing a customer's claim submission, verifying its validity against the policy coverage, and determining the payout amount

9. 2. Use case Description:

9 2. 1. Customer Use Cases:**Enroll in Policy**

Table 9.1: Enroll in Policy

Description	Allows a customer to enroll in a new insurance policy.
Actor	Customer
Preconditions	The customer is not currently enrolled in any policy.
Postconditions	A new policy record is created for the customer in the database.
Main Flow	1. Customer accesses the system. 2. Customer selects the desired policy type and coverage options. 3. Customer provides personal and medical information. 4. Customer submits the enrollment form. 5. System validates the information and creates a new policy record.
Alternative Flow	2a. If the provided information is incomplete or invalid, the system displays error messages and prompts the customer to correct the information before proceeding.

Table 9.1 shows the use case description of 'Enroll in Policy'. The "Enroll in Policy" process allows new customers to sign up for an insurance plan. First, the customer accesses the system and selects their preferred policy type and coverage options. They then provide personal and medical information. Once submitted, the system validates this information. If everything is valid, a new policy record is created for the customer in the database. However, if there are any errors or missing details, the system will display messages prompting the customer to correct the information before they can proceed.

Manage Policy

Table 9.2: Manage Policy

Description	Allows a customer to update their personal information, renew their policy, or modify policy coverage.
Actor	Customer
Preconditions	The customer has an existing active policy.
Postconditions	The customer's policy information is updated in the database.
Main Flow	1. Customer logs in to the system. 2. Customer navigates to the "Manage Policy" section. 3. Customer updates personal information, renews the policy, or modifies the policy coverage. 4. Customer saves the changes. 5. System updates the policy record in the database.
Alternative Flow	3a. If the customer attempts to modify the policy in a way that violates the policy terms or conditions, the system displays an error message and prevents the action.

Table 9.2 shows the use case description for 'Manage Policy'. Existing customers can manage their policies through the system's "Manage Policy" section. This allows them to update personal information, renew coverage for continued protection, or modify their plan's details. After logging in and navigating to this section, customers can make their desired changes and save them. The system will then verify the information and update the customer's policy record, although any modifications that violate policy terms will be blocked with an error message.

Submit Claim

Table 9.3: Submit Claim

Description	Allows a customer to submit a claim for reimbursement of medical expenses.
Actor	Customer
Preconditions	The customer has an active policy, and the medical expenses are covered under the policy.
Postconditions	A new claim record is created for the customer in the database.
Main Flow	1. Customer logs in to the system. 2. Customer navigates to the "Submit Claim" section. 3. Customer provides claim details, such as medical treatment information, receipts, and supporting documents. 4. Customer submits the claim. 5. System validates the claim and creates a new claim record in the database.
Alternative Flow	3a. If the customer tries to submit a claim for uncovered medical expenses or with incomplete information, the system displays an error message and prevents the submission.

Table 9.3 shows the use case description for ‘Submit Claim’.

The "Submit Claim" function allows customers to request reimbursement for covered medical expenses. Customers with active policies can access this section after logging in. There, they'll provide details about their claim, including medical treatment information, receipts, and any other relevant documents. Once submitted, the system validates the claim against the customer's policy and creates a new claim record if everything is in order. However, the system will prevent claim submission and display error messages if the requested expenses aren't covered by the policy or if the information provided is incomplete.

9. 2. 2. Hospital Staff Use Cases:

Admit Patient

Table 9.4: Admit Patient

Description	Allows hospital staff to admit a patient and verify their insurance coverage.
Actor	Hospital Staff
Preconditions	The patient has an active insurance policy.
Postconditions	A new patient record is created in the hospital system, and the patient's insurance coverage is verified.
Main Flow	1. Hospital staff accesses the system. 2. Hospital staff selects the option to admit a new patient. 3. Hospital staff enters the patient's information and insurance policy details. 4. System verifies the patient's insurance coverage. 5. Hospital staff completes the admission process.
Alternative Flow	4a. If the patient's insurance coverage cannot be verified or is invalid, the system displays an error message, and the staff may need to obtain additional information or process the admission without insurance coverage.

Table 9.4 shows Use case description of 'Admit Patient'. The "Admit Patient" function within the system streamlines patient registration for hospital staff. Staff can initiate the process by logging in and selecting the admit patient option. They then enter the patient's information alongside their insurance details. The system verifies the insurance coverage to ensure validity. Upon successful verification, the staff can finalize the admission process. However, if the insurance can't be verified, the system will notify staff who may then need to gather more information or proceed with admission without insurance coverage.

Perform Tests

Table 9.5: Perform Tests

Description	Allows hospital staff to order and perform tests for admitted patients.
Actor	Hospital Staff
Preconditions	The patient is admitted to the hospital.
Postconditions	Test records are created in the system, and the associated charges are added to the patient's bill.
Main Flow	1. Hospital staff accesses the patient's record. 2. Hospital staff selects the option to order tests. 3. Hospital staff selects the required tests and enters any additional information. 4. System creates test records and updates the patient's bill accordingly.
Alternative Flow	-

Table 9.5 shows Use case description of ‘Perform Tests’. Hospital staff can utilize the system to order and perform necessary tests on admitted patients. After accessing the patient's record, staff select the option to order tests. They can then choose the specific tests required and provide any additional details. The system efficiently creates records for these tests and automatically updates the patient's bill to reflect the associated charges.

Generate Bill

Table 9.6: Generate Bills

Description	Allows hospital staff to generate a bill for an admitted patient, including any applicable insurance coverage.
Actor	Hospital Staff

Description	Allows hospital staff to generate a bill for an admitted patient, including any applicable insurance coverage.
Preconditions	The patient is admitted to the hospital, and their insurance coverage has been verified.
Postconditions	A bill record is created in the system, including any applicable insurance coverage and deductions.
Main Flow	1. Hospital staff accesses the patient's record. 2. Hospital staff selects the option to generate a bill. 3. System calculates the total charges, including any tests, treatments, or services provided. 4. System applies any applicable insurance coverage and deductions. 5. Hospital staff reviews and finalizes the bill. 6. System creates a new bill record in the database.
Alternative Flow	-

Table 9.6 shows Use case description of 'Generate Bill'. The "Generate Bill" function within the system allows hospital staff to create a bill for admitted patients, factoring in their insurance coverage. Once staff access the patient record and confirm verified insurance, they can initiate bill generation. The system calculates the total charges for treatments, tests, and services provided. It then considers any applicable insurance coverage and deductions to determine the final amount. Staff have the opportunity to review and finalize the bill before a new record is created in the system.

Verify Insurance Coverage

Table 9.7: Verify Insurance Coverage

Description	Allows hospital staff to verify a patient's insurance coverage and eligibility.
Actor	Hospital Staff
Preconditions	The patient has provided their insurance policy details.
Postconditions	The patient's insurance coverage status is updated in the system.
Main Flow	1. Hospital staff accesses the patient's record. 2. Hospital staff selects the option to verify insurance coverage. 3. Hospital staff enters the patient's insurance policy details. 4. System communicates with the insurance provider to verify the coverage. 5. System updates the patient's record with the coverage status and details.
Alternative Flow	4a. If the insurance coverage cannot be verified or is invalid, the system displays an error message, and the staff may need to obtain additional information or process the admission without insurance coverage.

Table 9.7 shows Use case description of 'Verify Insurance Coverage'. The "Verify Insurance Coverage" function ensures patients' insurance validity. Hospital staff can initiate verification by accessing the patient record and selecting the dedicated option. After entering the patient's insurance details, the system communicates with the provider to confirm coverage. If successful, the patient's record is updated with the coverage status and details. However, if verification fails or the coverage is invalid, the system will notify staff who may then need to gather more information or proceed without insurance.

Update Patient Medical Records

Table 9.8: Update Patient Medical Records

Description	Allows hospital staff to update a patient's medical records, including test results and treatment information.
Actor	Hospital Staff
Preconditions	The patient is admitted to the hospital.
Postconditions	The patient's medical records are updated in the system.
Main Flow	1. Hospital staff accesses the patient's record. 2. Hospital staff selects the option to update medical records. 3. Hospital staff enters or updates the patient's medical information, test results, and treatment details. 4. Hospital staff saves the changes. 5. System updates the patient's medical records in the database.
Alternative Flow	-

Table 9.8 shows Use case description of ‘Update Patient Medical Records’. Hospital staff can leverage the system to update a patient's medical records throughout their stay. After accessing the patient's record, staff can select the option to update medical records. This allows them to enter or modify information regarding the patient's medical history, test results, and details about the treatment provided. Once the staff saves the changes, the system efficiently updates the patient's medical records within the database.

9. 2. 3. Insurance staff Use cases:

Process Enrolment

Table 9.9: Process Enrolment

Description	Allows insurance staff to process a new policy enrollment for a customer.
Actor	Insurance Staff
Preconditions	The customer has submitted an enrollment application.
Postconditions	A new policy record is created for the customer in the insurance system.
Main Flow	1. Insurance staff accesses the system. 2. Insurance staff selects the option to process a new enrollment. 3. Insurance staff reviews the customer's enrollment application and information. 4. Insurance staff verifies the information and eligibility criteria. 5. Insurance staff approves the enrollment. 6. System creates a new policy record for the customer in the database. 7. System generates and assigns a policy number and other relevant details. 8. Insurance staff notifies the customer about the successful enrollment and provides policy details.
Alternative Flow	4a. If the enrollment application or information is incomplete or invalid, the insurance staff may request additional information from the customer or reject the enrollment. 1. Insurance staff notifies the customer about the missing or invalid information. 2. Customer provides the requested information or updates the application. 3. If the updated information is valid, proceed to step 5. Otherwise, reject the enrollment. 4b. If the customer does not meet the eligibility criteria for the requested policy, the insurance staff rejects the enrollment. 1. Insurance staff notifies the customer about the ineligibility and provides reasons. 2. Insurance staff may suggest alternative policy options, if available. 3. If the customer agrees to an alternative policy, restart the enrollment process with the new policy details. 5a. If the enrollment is approved but there are additional requirements or documentation needed, the insurance staff requests the necessary information from the customer. 1. Insurance staff notifies the customer about the additional requirements. 2. Customer provides the requested information or documentation. 3. If the provided information is satisfactory, proceed to step 6. Otherwise, go back to step 4a.

Table 9.9 shows Use case description of ‘Process Enrollment’. Insurance staff utilize the system to process new customer enrollment applications. After accessing the system, staff review the submitted application and customer information. They then verify its accuracy and ensure the customer meets eligibility criteria for the chosen policy. If everything checks out, the enrollment is approved, triggering the system to create a new policy record, assign a policy number, and generate other relevant details. Staff then notify the customer of their successful enrollment and provide them with their policy information. However, if the application is incomplete, invalid, or the customer doesn't qualify, the staff will request additional information, suggest alternative plans, or reject the enrollment with an explanation. Even with approval, there might be additional requirements needed, and the system facilitates communication to ensure a smooth enrollment process.

Manage Policies

Table 9.10: Manage Policies

Description	Allows insurance staff to manage existing policies, including updates, renewals, and cancellations.
Actor	Insurance Staff
Preconditions	The customer has an existing policy in the insurance system.
Postconditions	The customer's policy record is updated in the database.
Main Flow	1. Insurance staff accesses the system. 2. Insurance staff searches for the customer's policy record. 3. Insurance staff performs the required action (update, renew, or cancel). 4. System updates the customer's policy record in the database.
Alternative Flow	-

Table 9.10 shows Use case description of ‘Manage Policies’. Insurance staff can manage existing customer policies through the system. This includes processing updates to policy details, renewals for continued coverage, and cancellations upon request. After logging in, staff can search for a specific customer's policy record. Once located, they can perform the desired action, whether it's updating information, renewing coverage, or cancelling the policy. The system will then make the necessary updates to the customer's policy record within the database.

Process Claims

Table 9.11. Process Claims

Description	Allows insurance staff to process claims submitted by customers.
Actor	Insurance Staff
Preconditions	The customer has submitted a claim, and their policy is active.
Postconditions	The claim record is updated with the processing status and outcome in the database.
Main Flow	1. Insurance staff accesses the system. 2. Insurance staff selects the option to process claims. 3. Insurance staff reviews the claim details and supporting documents. 4. Insurance staff verifies the claim against the customer's policy coverage. 5. Insurance staff approves or rejects the claim. 6. System updates the claim record in the database with the processing status and outcome.
Alternative Flow	4a. If the claim is incomplete or violates the policy terms, the insurance staff may request additional information from the customer or reject the claim.

Table 9.11 shows Use case description of ‘Process Claims’. Insurance staff leverage the system to handle customer claim submissions. After logging in and selecting the claims processing

option, staff meticulously review the claim details and any accompanying documents. They then verify if the claim falls under the customer's policy coverage. Upon verification, the staff approves the claim, triggering the system to update the claim record with a successful processing status. However, if the claim is incomplete, violates policy terms, or lacks necessary documentation, the staff may request additional information from the customer. In cases where the claim remains invalid, the system will reflect a rejection status within the claim record.

Chapter 10

System Evolution

We have created the tables and inserted values shown below for each entity.

Person:

```
CREATE TABLE Person (
```

```
    PersonID INT PRIMARY KEY,
```

```
    Name VARCHAR(255),
```

```
    DOB DATE,
```

```
    Status VARCHAR(50),
```

```
    PhoneNumber VARCHAR(20),
```

```
    Age INT,
```

```
    Password VARCHAR(255),
```

```
    Email VARCHAR(255),
```

```
    AddressID INT
```

```
);
```

```
INSERT INTO Person (PersonID, Name, DOB, Status, PhoneNumber, Age, Password,  
Email, AddressID)
```

```
VALUES
```

```
(1, 'John Doe', '1990-05-15', 'Active', '123-456-7890', 32, 'password123',  
'johndoe@example.com', 1),
```

```
(2, 'Jane Smith', '1985-09-20', 'Active', '987-654-3210', 39, 'pass1234',  
'janesmith@example.com', 2)
```

(3, 'Michael Johnson', '1978-03-10', 'Inactive', '555-123-4567', 46, 'mikepass', 'mjohnson@example.com', 3),

(4, 'Emily Davis', '1995-07-25', 'Active', '111-222-3333', 29, 'emilypass', 'emilydavis@example.com', 4),

(5, 'David Wilson', '1980-01-12', 'Active', '444-555-6666', 44, 'davidpass', 'dwilson@example.com', 5);

Policy:

CREATE TABLE Policy (

PolicyNumber INT PRIMARY KEY,

SumInsured DECIMAL(10, 2),

Premium DECIMAL(10, 2),

MedicalHistory VARCHAR(255),

PolicyBenefits VARCHAR(255),

Tenure INT,

PersonID INT,

FOREIGN KEY (PersonID) REFERENCES Person(PersonID)

);

INSERT INTO Policy (PolicyNumber, SumInsured, Premium, MedicalHistory, PolicyBenefits, Tenure, PersonID)

VALUES

(1001, 10000.00, 500.00, 'None', 'Hospitalization, Medications', 12, 1),

(1002, 500000.00, 2000.00, 'None', 'Death Benefit', 24, 2),

(1003, 30000.00, 500.00, 'None', 'Accident Coverage', 12, 3),

(1004, 10000.00, 300.00, 'None', 'Trip Cancellation, Medical Coverage', 6, 4),

(1005, 200000.00, 1000.00, 'None', 'Property Damage, Theft Coverage', 12, 5);

Disease:

CREATE TABLE Disease (

DiseaseCode INT PRIMARY KEY,

DiseaseName VARCHAR(100)

);

INSERT INTO Disease (DiseaseCode, DiseaseName)

VALUES

(101, 'Diabetes'),

(102, 'Hypertension'),

(103, 'Arthritis'),

(104, 'Asthma'),

(105, 'Depression');

Claims:

CREATE TABLE Claim (

ClaimID INT PRIMARY KEY,

PolicyNumber INT,

PersonID INT,

ClaimedAmount DECIMAL(10, 2),

IncurredAmount DECIMAL(10, 2),

Status VARCHAR(50),

```
    IssuedDate DATE,  
  
    FOREIGN KEY (PolicyNumber) REFERENCES Policy(PolicyNumber),  
  
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)  
  
);  
  
INSERT INTO Claim (ClaimID, PolicyNumber, PersonID, ClaimedAmount,  
IncurredAmount, Status, IssuedDate)  
  
VALUES  
  
    (2001, 1001, 1, 2000.00, 1500.00, 'Pending', '2023-12-10'),  
  
    (2002, 1002, 2, 100000.00, 95000.00, 'Approved', '2023-11-20'),  
  
    (2003, 1003, 3, 5000.00, 4500.00, 'Rejected', '2023-10-15'),  
  
    (2004, 1004, 4, 2000.00, 1800.00, 'Pending', '2023-12-28'),  
  
    (2005, 1005, 5, 10000.00, 9000.00, 'Approved', '2023-11-18');
```

Test:

```
CREATE TABLE Test (  
  
    TestCode INT PRIMARY KEY,  
  
    DiseaseCode INT,  
  
    TestName VARCHAR(100),  
  
    Price DECIMAL(10, 2),  
  
    FOREIGN KEY (DiseaseCode) REFERENCES Disease(DiseaseCode)  
  
);  
  
INSERT INTO Test (TestCode, DiseaseCode, TestName, Price)  
  
VALUES
```


(301, 101, 'Blood Sugar Test', 50.00),
(302, 102, 'Blood Pressure Test', 30.00),
(303, 103, 'Joint Function Test', 75.00),
(304, 104, 'Lung Function Test', 100.00),
(305, 105, 'Mental Health Evaluation', 150.00);

Billing:

CREATE TABLE Billing (

BillID INT PRIMARY KEY,

DateOfDischarge DATE,

DateOfAdmission DATE,

BillAmount DECIMAL(10, 2),

DiseaseCode INT,

FOREIGN KEY (DiseaseCode) REFERENCES Disease(DiseaseCode)

);

INSERT INTO Billing (BillID, DateOfDischarge, DateOfAdmission, BillAmount,
DiseaseCode)

VALUES

(4001, '2023-12-10', '2023-12-05', 1500.00, 101),
(4002, '2023-11-20', '2023-11-15', 95000.00, 102),
(4003, '2023-10-15', '2023-10-10', 4500.00, 103),
(4004, '2023-12-28', '2023-12-20', 1800.00, 104),
(4005, '2023-11-18', '2023-11-10', 9000.00, 105);

Address:

```
CREATE TABLE Address (  
  
    AddressID INT PRIMARY KEY,  
  
    State VARCHAR(50),  
  
    City VARCHAR(50),  
  
    PinCode VARCHAR(10)  
  
);  
  
INSERT INTO Address (AddressID, State, City, PinCode)  
  
VALUES  
  
    (1, 'California', 'Los Angeles', '90001'),  
  
    (2, 'New York', 'New York City', '10001'),  
  
    (3, 'Texas', 'Houston', '77001'),  
  
    (4, 'Florida', 'Miami', '33101'),
```

Hospital:

```
CREATE TABLE Hospital (  
  
    HospitalID INT PRIMARY KEY,  
  
    Name VARCHAR(100),  
  
    HospitalAddress VARCHAR(255),  
  
    AddressID INT,  
  
    FOREIGN KEY (AddressID) REFERENCES Address(AddressID)  
  
);  
  
INSERT INTO Hospital (HospitalID, Name, HospitalAddress, PersonIDID)
```

VALUES

(5001, 'General Hospital', '123 Main St, Los Angeles', 1),
(5002, 'City Medical Center', '456 Oak St, New York City', 2),
(5003, 'Houston General Hospital', '789 Maple Ave, Houston', 3),
(5004, 'Sunshine Hospital', '789 Palm St, Miami', 4);

Chapter 11

Testing

Table 11.1 Test Cases

SL.No.	Test Case	Feature Name	Sub Feature	Test case Type	Procedure	Expected Result
1	Add new Customer	Customer Management	New Customer Enrollment	Positive	1. Access the system as an admin user. 2. Navigate to the 'Customer Management' section. 3. Click on the 'Add New Customer' button. 4. Enter valid personal information (Name, DOB, Phone number, Age, Email, Password). 5. Submit the form.	The system should successfully create a new customer record in the database with the provided details.
2	Add New Customer	Customer Management	New Customer Enrollment	Negative	1. Access the system as an admin user. 2. Navigate to the	The system should display appropriate error messages and prevent the creation of a new customer

					'Customer Management' section. 3. Click on the 'Add New Customer' button. 4. Leave required fields (e.g., Name, DOB) empty. 5. Submit the form.	record until all required fields are filled correctly.
3	Enroll in Policy	Policy Management	New Policy Enrollment	Positive Test Case	1. Log in as an existing customer. 2. Navigate to the 'Enroll in Policy' section. 3. Select the desired policy type and coverage options. 4. Enter required information (Sum Insured,	The system should successfully create a new policy record in the database associated with the customer, and store the provided policy details.

					Premium, Medical History, Policy Benefits, Tenure). 5. Submit the enrollment form.	
4	Update Medical History	Policy Management	Update Policy Details	Positive Test Case	1. Log in as an existing customer with an active policy. 2. Navigate to the 'Update Medical History' section. 3. Update the medical history information. 4. Save the changes.	The system should successfully update the customer's medical history in the associated policy record.
5	Submit Claim	Claim Management	New Claim Submission	Positive Test Case	1. Log in as an existing customer with an active policy.	The system should successfully create a new claim record in the database, associated with the

					2. Navigate to the 'Submit Claim' section. 3. Enter claim details (Policy Number, Person ID, Claimed Amount, Incurred Amount, Status, Issued Date). 4. Attach relevant documents or receipts. 5. Submit the claim.	customer's policy and provided claim details.
6	Track Claim Status	Claim Management	Claim Status Tracking	Positive Test Case	1. Log in as an existing customer with a submitted claim. 2. Navigate to the 'Track Claim' section.	The system should display the current status and details of the claim, such as whether it's pending, approved, or rejected.

					3. Search for the claim using the Claim ID or other relevant filters.	
7	Add Test	Test Management	New Test Addition	Positive Test Case	1. Access the system as a hospital staff user. 2. Navigate to the 'Test Management' section. 3. Click on the 'Add New Test' button. 4. Enter test details (Test Code, Disease Name, Test Name, Price, Hospital ID). 5. Submit the form.	The system should successfully create a new test record in the database with the provided details.
8	Generate Bill	Billing Management	Bill Generation	Positive Test Case	1. Access the system as a hospital staff user.	The system should successfully create a new billing record

					2. Navigate to the 'Billing Management' section. 3. Select the patient and associated claim. 4. Enter billing details (Date of Discharge, Date of Admission, Bill Amount, Disease Code, Bill ID). 5. Generate the bill.	in the database, associated with the patient's claim and provided billing details.
9	Update Hospital Details	Hospital Management	Update Hospital Information	Positive Test Case	1. Access the system as an admin user. 2. Navigate to the 'Hospital Management' section. 3. Search for the	The system should successfully update the hospital's information in the database with the provided details.

					hospital by Hospital ID or Name. 4. Update the hospital details (Name, Address). 5. Save the changes.	
10	Add New Disease	Disease Management	New Disease Addition	Positive Test Case	1. Access the system as an admin user. 2. Navigate to the 'Disease Management' section. 3. Click on the 'Add New Disease' button. 4. Enter the disease details (Disease Code, Disease Name). 5. Submit the form.	The system should successfully create a new disease record in the database with the provided details.

Chapter 12

Results

12.1 Find all people with active status.

SQL Query:

```
SELECT *
FROM Person
WHERE Status = 'Active';
```

Relational algebra query:

$\pi(\text{PersonID}, \text{Name}, \text{DOB}, \text{Status}, \text{PhoneNumber}, \text{Age}, \text>Password, \text>Email}, \text{AddressID}) (\sigma(\text{Status} = \text{'Active'}) (\text{Person}))$

Output:

Table 12.1 Output 1

Name	Email	PhoneNumber	Age
John Doe	johndoe@example.com	123-456-7890	32
Jane Smith	janesmith@example.com	987-654-3210	39
Emily Davis	emilydavis@example.com	111-222-3333	29
David Wilson	dwilson@example.com	444-555-6666	44

12.2 Find all policies with a sum insured greater than 100,000.

SQL Query:

```
SELECT *
FROM Policy
WHERE SumInsured > 100000;
```

Relational algebra query:

$\pi(\text{PolicyNumber}, \text{SumInsured}, \text{Premium}, \text{MedicalHistory}, \text{PolicyBenefits}, \text{Tenure}, \text{PersonID}) (\sigma(\text{SumInsured} > 100000) (\text{Policy}))$

Output:

Table 12.2 Output 2

PolicyNumber	SumInsured	Premium	MedicalHistory	Tenure	PersonID
1002	500000	2000	None	24	2

12.3 Find the names and diseases of people with bills related to Arthritis (DiseaseCode 103)**SQL Query:**

```
SELECT p.Name, d.DiseaseName
FROM Person p
INNER JOIN Billing b ON p.PersonID = b.PersonID
INNER JOIN Disease d ON b.DiseaseCode = d.DiseaseCode
WHERE b.DiseaseCode = 103;
```

Relational algebra query:

$$\pi(p.Name, d.DiseaseName) (Person\ p\ JOIN\ Billing\ b\ ON\ p.PersonID = b.PersonID\ JOIN\ Disease\ d\ ON\ b.DiseaseCode = d.DiseaseCode\ (\sigma(DiseaseCode = 103)(Billing)))$$
Output:

Table 12.3 Output 3

Name	DiseaseName
Michael Johnson	Arthritis

12.4 Find people with active status and their corresponding policy details (PolicyNumber, SumInsured).**SQL Query:**

```
SELECT p.Name, p.Email, po.PolicyNumber, po.SumInsured
FROM Person p
INNER JOIN Policy po ON p.PersonID = po.PersonID WHERE p.Status = 'Active';
```

Relational algebra query:

```
 $\pi(p.Name, p.Email, po.PolicyNumber, po.SumInsured)$ 
(Person p JOIN Policy po ON p.PersonID = po.PersonID
( $\sigma$ (Status = 'Active')(Person)))
```

Output:

Table 12.4 Output 4

Name	Email	PolicyNumber	SumInsured
John Doe	johndoe@example.com	1001	10000
Jane Smith	janesmith@example.com	1002	500000
Emily Davis	emilydavis@example.com	1004	10000
David Wilson	dwilson@example.com	1005	200000

12.5. Find the total bill amount per disease for bills discharged in December 2023.

SQL Query:

```
SELECT d.DiseaseName, SUM(BillAmount) AS TotalBillAmount
FROM Billing b
INNER JOIN Disease d ON b.DiseaseCode = d.DiseaseCode
WHERE YEAR(DateOfDischarge) = 2023 AND MONTH(DateOfDischarge) = 12
GROUP BY d.DiseaseName;
```

Relational algebra query:

$\pi(d.DiseaseName, SUM(b.BillAmount) \text{ AS } TotalBillAmount)$
 $((Billing \text{ b JOIN Disease d ON b.DiseaseCode} = d.DiseaseCode)$
 $(\sigma(YEAR(DateOfDischarge) = 2023 \text{ AND MONTH(DateOfDischarge)} = 12)(Billing)))$
 $GROUP \text{ BY } d.DiseaseName$

Output:

Table 12.5 Output 5

DiseaseName	TotalBillAmount
Diabetes	1500
Asthma	1800

12.6 Find all people with claims exceeding \$5,000 and their current policy details (PolicyNumber, SumInsured).

SQL Query:

SELECT p.Name, p.Email, c.ClaimID, c.ClaimedAmount, po.PolicyNumber, po.SumInsured
 FROM Person p
 INNER JOIN Claim c ON p.PersonID = c.PersonID
 INNER JOIN Policy po ON p.PersonID = po.PersonID
 WHERE c.ClaimedAmount > 5000;

Relational algebra query:

$\pi(p.Name, p.Email, c.ClaimID, c.ClaimedAmount, po.PolicyNumber, po.SumInsured)$
 $(Person \text{ p JOIN Claim c ON p.PersonID} = c.PersonID$
 $JOIN Policy po ON p.PersonID = po.PersonID$

$\sigma(c.ClaimedAmount > 5000)(Claim)$

Output:

Table 12.6 Output 6

Name	Email	ClaimID	ClaimedAmount	PolicyNumber	SumInsured
Jane Smith	janesmith@example.com	2002	100000	1002	500000

Chapter 13

Conclusion

In conclusion, this project has successfully developed and documented a Healthcare Insurance Management System (HIMS) using a robust Database Management System (DBMS). The HIMS offers a wide range of functionalities to automate and optimize healthcare insurance operations, including policy management, claims processing, member information handling, and billing. This translates into improved efficiency and accuracy in managing healthcare insurance policies and claims, ultimately benefiting both insurers and policyholders. The report delves into the core components of the system, highlighting the well-structured database design with its entities, attributes, and relationships. It also discusses the chosen DBMS and its functionalities, showcasing its suitability for managing sensitive healthcare data securely. This project serves as a valuable example of applying DBMS concepts to a real-world healthcare scenario. By streamlining key processes, the HIMS has the potential to significantly enhance healthcare insurance operations and improve customer service. Future endeavors may explore additional features or integration with healthcare systems for a more comprehensive solution.